COEN320
Intro to Real-Time Systems

Project Report

Written By:
Maxime Pagé 40114955
p_maxime@live.concordia.ca
Simon Guindon 40058301
s_uindon@live.concordia.ca
Patrick Han 40112063
pa_an@live.concordia.ca

Due Date: 07/04/2025

"We certify that this submission is our original work and meets the Faculty's Expectations of Originality."

I - Objectives

The objective of this project is to design and implement a simplified version of a real-time system for an Air Traffic Monitoring and Control (ATC) system. We are using a QNX real-time operating system, written in C++. To do so, we will use Inter-Process communication and System Architectures as follows : Our system will be implemented as 6 distincts QNX executables, one for each major subsystem, and will use shared memory (using the writer/reader template) and a client/server relationship.

II - Introduction

A - Definitions

The following will contain the relevant background information about our project so that someone outside of the course can understand the context. An ATC system is a **real-time system** that is in charge of regulating air traffic flow. It involves collision prevention, reasonable separation between aircrafts and safe trajectories. A system that we call “real-time” is one that responds to events that are happening at any moment. A simple example of a real-time system is an elevator; the elevator doors must open if someone presses a button, and they must close if the elevator is changing floors.
The ATC that are implemented in real life are extremely complex and involve a lot of different mechanisms and subsystems to control air flow from the departure from the gates all the way to the arrival at the gates of their destination. We can differentiate these as different sectors called the Tower Control, the Approach and the En-Route. See figure 1 below.

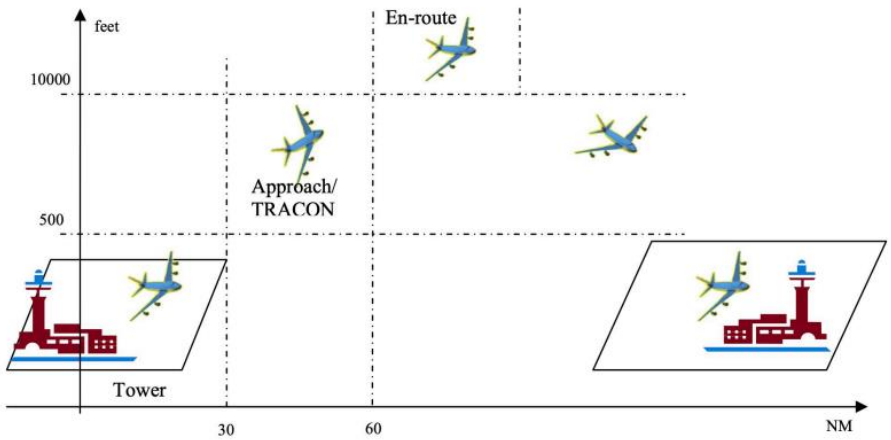


Fig. 1: Air traffic control system

Since the goal of this project is to create a simplified version of the ATC, we will be focusing on the En-Route sector, which is in other words the area in which the aircrafts have reached cruising speed and altitude (basically the bulk of the flight). Therefore we are dealing with an area 10,000ft by 10,000ft by 25,000ft at an altitude of 15,000ft. See Figure 2 below.

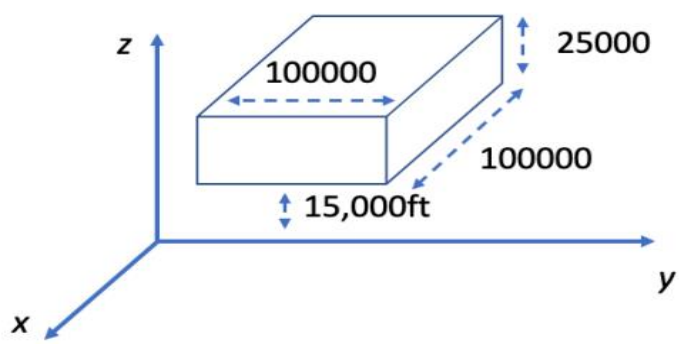


Fig. 2. 3D airspace

B – Requirements & Assumptions

There are 5 main requirements for this project:

1) Every **5 seconds**, a **plan view** (i.e., a top-down view) of the airspace must be displayed, showing the **current position of every aircraft**. Note: during coding, we found that a 3 seconds refresh rate provided better representation of the aircraft flow. 5 seconds felt too long during execution and below 3 seconds was too fast, so we compromised on a middle ground at 3 seconds.

2) The **controller** (i.e., the person running the simulation) must be able to adjust a parameter **n** at runtime. This parameter enables the system to check, at **[current_time + n] seconds**, whether **high congestion** is likely. In essence, it provides a **future projection** of each aircraft's trajectory to identify potential **violations of separation constraints**. For thoroughness's sake, the way we went about solving this problem was to always use the highest required value for the prediction model (the guidelines for the assignment suggested that a visual alarm must inform the operator if a safety violation occurs in the next 2 minutes, so we predicted the next 120 seconds at every second). Therefore, our computer system checks if every plane is within a dangerous distance of another plane in the next 2 minutes and will ask the operator if they want to intervene and notify the aircrafts.

3) If a **separation violation** is predicted to occur within the next **2 minutes**, an **alarm** must be triggered, either **visually** or **audibly**.
4) Every **20 seconds**, a snapshot of the entire airspace must be **saved to a file**. This allows for a **complete historical reconstruction** of the airspace.

5) Every **request and command** issued by the **operator** must be recorded in a dedicated **log file**.
Here are the additional assumptions we made during the implementation of the project.

Firstly, we are assuming that a plan view is a top-down, 2D projection of the airspace for the DataDisplay, but the Aircraft will be listed above the 2D grid with their respective Z position to contain all relevant information. We also assumed that the simulation ran for 90 seconds (could be changed by changing the values in the execution for loops of the processes, if required.)

Another very important assumption that we made, which is crucial for the solution we implemented, is that the “normal” speed is Z direction is 0 by default. This was done for two main reasons: 1) It simulates realistic flight where airplanes follow a steady altitude while in the air (as in the planes are not constantly ascending nor descending for their entire travel through the flight zone). 2) Altering the Z-speed is the core of our solution to solve the safety violation between two aircrafts. The in-depth implementation will be discussed in Design & Implementation.

Another assumption that we had to make, which was not explicitly mentioned in the guideline, is how the “various operating conditions (low medium high and overloads)” are defined. As such, we estimated that a low traffic was spawning 5 Aircrafts, medium traffic was spawning 15 Aircrafts, high traffic was spawning 30 Aircrafts, overload traffic was spawning 50 Aircrafts. Anything more was getting very confusing to look at on the console output of the processes, and too few aircrafts were insufficient to properly test our solution.

We also assumed that the Aircrafts could either “spawn” directly inside the zone, or slightly outside and “cross into” the zone during runtime. This can be seen by looking at our 50 switch case statement for every plane we created inside the Aircraft.cpp file.

III - Analysis

In this section, we will discuss our understanding of each requirement.

1) **Update Aircraft Positions –** Every **3 seconds**, display an updated **plan view** of the airspace, showing the current position of all aircrafts.

- Since a plan view is a 2D projection, we only need the X and Y coordinates to show the position of the aircrafts. The Z coordinate will still be shown as a separate element.

2) **Adjustable Future Prediction –** Check all aircrafts in the airspace at the current time + n seconds for constraint violations. The controller must be able to adjust a parameter **n** at runtime. This determines how far ahead (e.g., 2 seconds or 50 seconds) the system projects aircraft trajectories, allowing us to anticipate potential congestion or violations of separation constraints.

- There must be a function with a parameter n that allows the operator to anticipate potential collisions. The controller is in charge of adjusting this parameter.
- From our understanding, when the Computer System declares a safety violation, the Controller will be asked if he wants to intervene and ask the Aircrafts to vary their Z position to avoid collision. Upon receiving the agreement from the Console Operator to act to solve the collision, the Computer System will update the Communication System process so that it can forward instructions to the specific aircrafts that need to shift their Z position by altering their Z speed for a few seconds.
- IMPORTANTLY: when dealing with dozen of aircrafts at once, manually responding to all the collision was creating a funnel effect, so we implemented an “auto-responder” to help fasten the response. This will be discussed further in the Operator section.

3) **Safety Violation Alerts –** If a **separation violation** (such as an imminent collision or dangerously close aircraft) is predicted within the next 2 minutes, trigger an **alarm**, either visually or audibly, to alert the operator.

- Using requirement number 2, we can create a function that, when we detect a constraint violation, an alarm will be implemented visually.

4) **Logging Airspace History –** Every 20 seconds, store the complete **airspace state** in a file. This allows the system to **reconstruct** past events and track aircraft movements over time.

- Similar to how the radar functions, the airspace logging can be easily implemented and the history can be stored in a text file.

5) **Logging Operator Actions –** Record every **command or request** made by the operator in a log file. For example, if the operator instructs an aircraft to reduce speed by 50 ft/s, the system must log that action along with the exact time it was issued.

- Similar to requirement number 4, every time the operator enters an input, we can implement a function that saves that input in a text file with the time and destination.

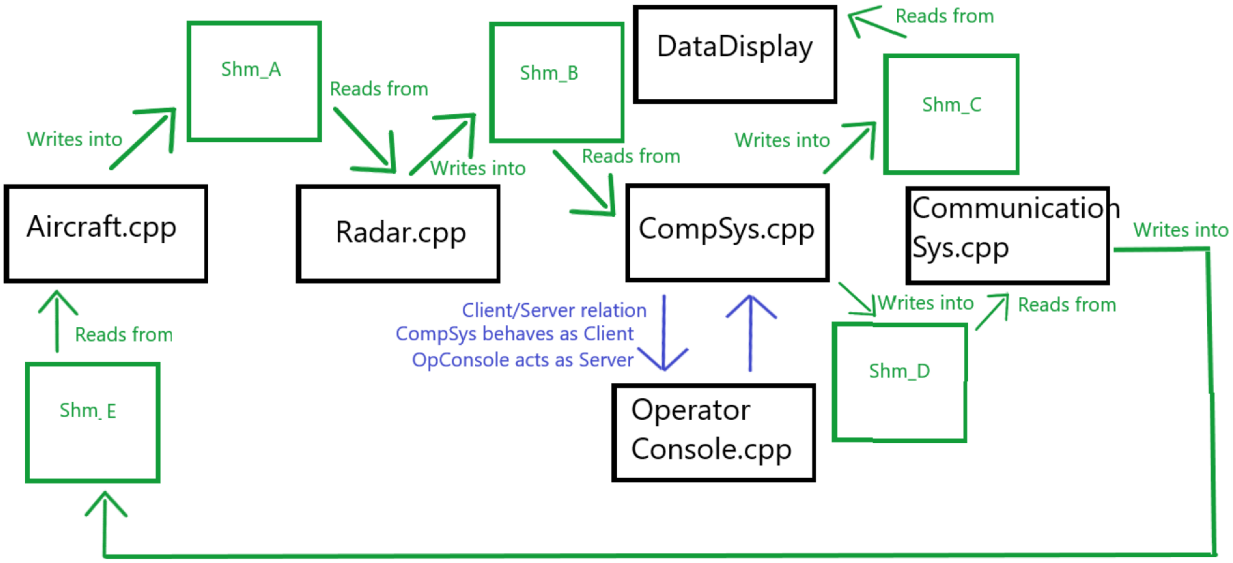
Design

A - Overview of our plan

The first thing we had to tackle for the solution was how to share the data between different processes in a way that would not cause chronological issues. As such, we decided on the following order of action: Every aircraft gets created in Aircraft.cpp and take turns writing in shared memory A and writing to the console, with a sleep(1) statement so that all aircrafts will print to the console and update the shared memory every second. Upon all threads finishing their writing in Shm_A, a semaphore_post() contacts Radar.cpp who was waiting at its semaphore_wait() statement.

In the same way, Radar will transfer through shared memory (Shm_B) the data to ComputerSystem, then sleep for 3 sections before waiting at the sem_wait() for the Aircrafts to finish writing. ComputerSystem is also timed with Radar because it also has a sleep (3) statement that will make him update every three seconds. No matter the outcome of the safety violation, CompSys will transfer the flight information to DataDisplay (Shm_C) so that it can print the 2D grid to the console.

In addition, if a safety violation occurs, it must notify the operator and wait for a response. This was done through the server/client template provided in the tutorials. Finally, upon receiving the green light from the operator, the Computer System will update its shared memory and trigger flags to forward the message to Communication System (Shm_D). In essence, Communication System will receive which aircrafts (identified by their id) will need to be notified so that they can set flags inside the Aircraft threads to ascend or descend (Shm_E). This feedback loop will resolve the collision that the Computer System found in a dynamic fashion.



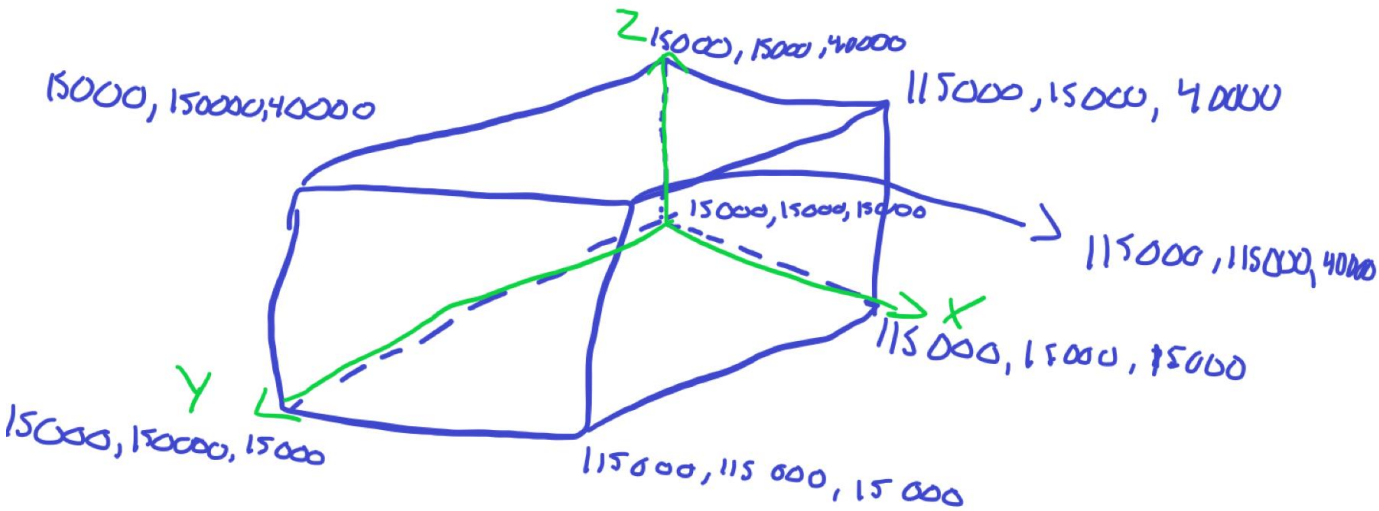
Implementation

A - Aircraft.cpp

The first component that was designed in this project was the Aircraft.cpp. To begin with, we implemented a structure for Aircraft containing a X, Y, Z value for speed, initial position and current position, as well as parameters to ascend and descend (will be discussed later in Communication Systems).
We also implemented a function to print the timestamp to the console for debugging and logging purposes. This Aircraft process uses two instances of shared memory. In the first instance, it behaves as a writer (and Radar, discussed below, will behave as a reader) and in the second instance it behaves as a reader (and the Communication System will behave as a writer). Finally, to have control over the flight we decided to implement a switch case where our 50 aircrafts can be tailored to have specific flight patterns in the airspace limited by the following coordinates:

15 000, 15 000, 15 000
15 000, 115 000, 15 000
115 000, 15 000, 15 000

115 000, 115 000, 15 000
15 000, 15 000, 40 000
15 000, 115 000, 40 000
115 000, 15 000, 40 000
115 000, 115 000, 40 000
Below is a representation of our flight zone



In order to correctly represent a system where different flights head in different directions, we decided that flights would start in either North, North East, East, South East, South, South West, West, North West at either Low altitude (20 000 to 30 000), Medium altitude (30 000 to 37 000) or high altitude (37 000 to 40 000). Once a starting position was decided, we would make them travel to the opposed “face” of the “face” they came from. For example, a plane spawning south west (low X value, low Y value) could either head directly north, north east or east (with positive X or Y speeds). One assumption that we chose in the design was that flights were travelling at a constant Z level, and would change their height position if a collision was detected. This would push flights to higher and lower levels and resolve the collision warning. Finally, another assumption that we decided was “how much Aircrafts are equivalent to low, medium, high and overload loads.” We decided arbitrarily to go with 5, 15, 30 and 50 respectively because anything above 50 would be hard to keep track of in the operator console (and Radar and Data display…)

The next step was to initialize the shared memory elements between the Aircrafts

& Radar, and CommunicationSystems and Aircrafts. Every aircraft is created with its own thread, where each Aircraft has its own id and knows how many aircrafts are present. The threads are joined with a join statement to make sure they all complete before the program finishes executing. In the main simulation loop, a given aircraft thread gets assigned a position and a speed and will first check if the Communication system has sent any updates (This is initially set to false.) If the thread sees that the communication system has updated flags, a given aircraft will update its flags ascend and descend. The logic that was used is to set the speed in Z to +100 for the aircraft with the lower id (to prevent deadlock) and -100 to the aircraft with the higher id. This would occur for 10 cycles of 1 second, so the one aircraft would move 1000 units upwards and the other would move 1000 downwards to avoid the collision. Finally, the aircraft thread will update its current position given its speed for a coordinate (so X speed increments X position and so on…). Then we used a mutex so that the threads would print to the console in order to avoid race conditions. An additional check was made so that every 20 seconds (with i of the main loop's modulus 20 is 0), the flight logs of every aircraft would be stored in a text file inside the ubuntu machine called flight_logs.txt. To make sure the program is running as intended, the console outputs every second the position and speed for any aircrafts.

```
[06/04/2025 - 03:31:45] Aircraft 0 Position: (17000, 37600, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 03:31:45] Aircraft 1 Position: (11800, 133200, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:45] Aircraft 2 Position: (11800, 143200, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:45] Aircraft 3 Position: (118200, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 03:31:45] Aircraft 4 Position: (17000, 94400, 19000) | Speed: (0, 900, 0)
[06/04/2025 - 03:31:46] Aircraft 0 Position: (17000, 36700, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 03:31:46] Aircraft 1 Position: (10600, 134400, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:46] Aircraft 2 Position: (10600, 144400, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:46] Aircraft 3 Position: (119400, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 03:31:46] Aircraft 4 Position: (17000, 95300, 19000) | Speed: (0, 900, 0)
[06/04/2025 - 03:31:47] Aircraft 0 Position: (17000, 35800, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 03:31:47] Aircraft 1 Position: (9400, 135600, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:47] Aircraft 2 Position: (9400, 145600, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:47] Aircraft 3 Position: (120600, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 03:31:47] Aircraft 4 Position: (17000, 96200, 19000) | Speed: (0, 900, 0)
[06/04/2025 - 03:31:48] Aircraft 0 Position: (17000, 34900, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 03:31:48] Aircraft 1 Position: (8200, 136800, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:48] Aircraft 2 Position: (8200, 146800, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:48] Aircraft 3 Position: (121800, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 03:31:48] Aircraft 4 Position: (17000, 97100, 19000) | Speed: (0, 900, 0)
[06/04/2025 - 03:31:49] Aircraft 0 Position: (17000, 34000, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 03:31:49] Aircraft 1 Position: (7000, 138000, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:49] Aircraft 2 Position: (7000, 148000, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 03:31:49] Aircraft 3 Position: (123000, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 03:31:49] Aircraft 4 Position: (17000, 98000, 19000) | Speed: (0, 900, 0)
```

B - Radar.cpp

The second component that was designed in this project was the Aircraft.cpp. To begin with, we implemented a zone that would be recorded by the Radar, according to the 8 coordinates explained section A. All aircrafts outside of this zone were not considered by the radar. In essence, the Radar.cpp serves as an interface between Aircraft.cpp and ComputerSystem.cpp. When interacting with Aircraft.cpp, every thread first writes in the shared memory, then uses a semaphore to signal Radar.cpp that the data is properly written and can be read. The Radar then reads from the shared memory, and forwards it by writing it into a second instance of shared memory it will share with ComputerSystem. Therefore, Radar acts both as a writer and a reader, with ComputerSystem and Aircraft processes respectively. To mimic the interaction of only spotting Aircrafts inside the designated flight zone, a check is made when reading from the shared memory to see if the aircraft is currently between X 15 000 and 115 000, Y 15 000 and 115 000 and Z 15 000 and 40 000. Now is a good time to talk about one drawback of our Aircraft control system: Upon starting the processes, I must manually enter which load will be used for the following processes: Aircraft, Radar, ComputerSystem, CommunicationSystem and DataDisplay. Our entire program will take for granted that the SAME option will be selected for all processes at the start of execution. Finally, for debugging purposes and to make sure that the code was reliably built, the radar prints into the console.

The last point to cover on this process is how it behaves control-wise to time itself with Aircraft.cpp and ComputerSystem. In Aircraft.cpp, a sem_post occurs at the end of the for loop, followed by a sleep(1) statement. In Radar.cpp, the sem_wait() statement will wait for the semaphore to be posted and a sleep(3) statement is put at the end of the execution for loop, along the sem_post() for ComputerSystem. As such, Aircraft can update independently its position 3 times (because of sleep(1)) and Radar will check the updated shared memory every 3 seconds.

Aircraft's sample console output (low density, 5 Aircrafts) - updates every 1 second

```
[06/04/2025 - 16:51:17] Aircraft 0 Position: (17000, 105100, 21000) | Speed: (0, -900, 100)
[06/04/2025 - 16:51:17] Aircraft 1 Position: (101800, 43200, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:17] Aircraft 2 Position: (101800, 53200, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:17] Aircraft 3 Position: (28200, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 16:51:17] Aircraft 4 Position: (17000, 26900, 19000) | Speed: (0, 900, -100)
[06/04/2025 - 16:51:18] Aircraft 0 Position: (17000, 104200, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 16:51:18] Aircraft 1 Position: (100600, 44400, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:18] Aircraft 2 Position: (100600, 54400, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:18] Aircraft 3 Position: (29400, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 16:51:18] Aircraft 4 Position: (17000, 27800, 19000) | Speed: (0, 900, 0)
[06/04/2025 - 16:51:19] Aircraft 0 Position: (17000, 103300, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 16:51:19] Aircraft 1 Position: (99400, 45600, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:19] Aircraft 2 Position: (99400, 55600, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:19] Aircraft 3 Position: (30600, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 16:51:19] Aircraft 4 Position: (17000, 28700, 19000) | Speed: (0, 900, 0)
[06/04/2025 - 16:51:20] Aircraft 0 Position: (17000, 102400, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 16:51:20] Aircraft 1 Position: (98200, 46800, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:20] Aircraft 2 Position: (98200, 56800, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:20] Aircraft 3 Position: (31800, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 16:51:20] Aircraft 4 Position: (17000, 29600, 19000) | Speed: (0, 900, 0)
```

Radar's sample console output (low density, 5 Aircrafts) - updates every 3 seconds

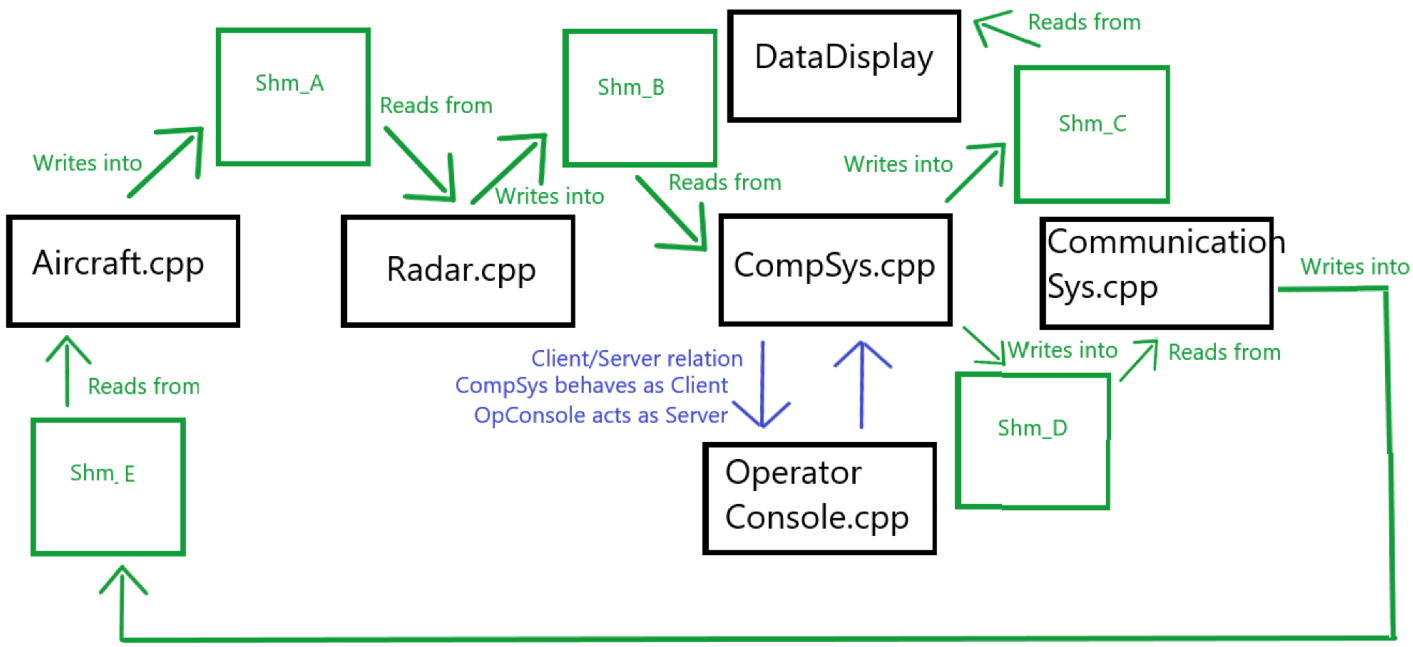
```
[06/04/2025 - 16:51:16] Received Data From Aircraft 0 -> Position: (17000, 106000, 20900) | Speed: (0, -900, 100)
[06/04/2025 - 16:51:16] Received Data From Aircraft 1 -> Position: (103000, 42000, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:16] Received Data From Aircraft 2 -> Position: (103000, 52000, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:16] Received Data From Aircraft 3 -> Position: (27000, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 16:51:16] Received Data From Aircraft 4 -> Position: (17000, 26000, 19100) | Speed: (0, 900, -100)
[06/04/2025 - 16:51:19] Received Data From Aircraft 0 -> Position: (17000, 103300, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 16:51:19] Received Data From Aircraft 1 -> Position: (99400, 45600, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:19] Received Data From Aircraft 2 -> Position: (99400, 55600, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:19] Received Data From Aircraft 3 -> Position: (30600, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 16:51:19] Received Data From Aircraft 4 -> Position: (17000, 28700, 19000) | Speed: (0, 900, 0)
[06/04/2025 - 16:51:22] Received Data From Aircraft 0 -> Position: (17000, 101500, 21000) | Speed: (0, -900, 0)
[06/04/2025 - 16:51:22] Received Data From Aircraft 1 -> Position: (97000, 48000, 25000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:22] Received Data From Aircraft 2 -> Position: (97000, 58000, 30000) | Speed: (-1200, 1200, 0)
[06/04/2025 - 16:51:22] Received Data From Aircraft 3 -> Position: (33000, 50000, 35000) | Speed: (1200, 0, 0)
[06/04/2025 - 16:51:22] Received Data From Aircraft 4 -> Position: (17000, 30500, 19000) | Speed: (0, 900, 0)
```

Finally, ComputerSystem is waiting on its sem_wait() statement for Radar to sem_post() at the end of the execution for loop.

C - ComputerSystem.cpp

The third component that was designed in this project, and the most complicated and important one, was the ComputerSystem.cpp. Due to the complexity of ComputerSystem's role in the program, the figure shown below

highlights all interprocess communications that were used:



In total, ComputerSystem has four communication channels with other processes, namely it will (1) read from SharedMemory B, (2) write in SharedMemory C, (3) write into SharedMemory D, and (4) behave as a client in a Client/Server relationship with OperatorConsole.cpp. Due to the fact that many interprocess communication happens here, it is important that our process is tightly regulated to ensure proper flow of information across time. Firstly, the Computer System cannot start operating unless it gets its go signal from the Radar.cpp class.

```
for (int i = 0; i < 30; ++i) {
    sem_wait(sem_radar);
    printTimestamp();
    std::cout << "Processing Radar data...\n";
    calculateProjections(projections, radarData, numAircrafts);
    detectCollisions(projections, numAircrafts, coid, commData, sem_sync);
    writeToDataDisplay(radarData, displayData, numAircrafts);
    sem_post(sem_display);
    printTimestamp();
    std::cout << "Data written to DataDisplay shared memory\n";
}
```

Once it gets the authorization to proceed, it enters the CalculateProjections() [shown below] function and will anticipate the next 120 seconds of flight for every aircraft by using a nested for loop.

```
printTimestamp();
std::cout << "Calculating position projections...\n";
for (int id = 0; id < numAircrafts; ++id) {
    for (int t = 0; t < 120; ++t) { // 120 seconds (2 minutes)
        int x = radarData[id * 6 + 0] + radarData[id * 6 + 3] * t;
        int y = radarData[id * 6 + 1] + radarData[id * 6 + 4] * t;
        int z = radarData[id * 6 + 2] + radarData[id * 6 + 5] * t;
        projections[id][t] = std::make_tuple(x, y, z);
    }
}
printTimestamp();
std::cout << "Position projections completed.\n";
```

Once the 2D vector is filled up with the data, we used a triple for loop so that for every second of flight for the next 120 seconds, the position of every aircraft would be compared to every other aircraft.

```
void detectCollisions(std::vector<std::vector<std::tuple<int, int, int>>> &projections, int numAircrafts, int coid, shared_data *commData, sem_t *
    printTimestamp();
    std::cout << "Checking for collisions...\n";
    for (int t = 0; t < 120; ++t) { // Check each second
        for (int id1 = 0; id1 < numAircrafts; ++id1) {
            for (int id2 = id1 + 1; id2 < numAircrafts; ++id2) {
                int x1, y1, z1, x2, y2, z2;
                std::tie(x1, y1, z1) = projections[id1][t]; //assign the tuple (X Y Z positions) the value of a given aircraft at a given time
                std::tie(x2, y2, z2) = projections[id2][t]; //assign the tuple (X Y Z positions) the value of a given aircraft at a given time

                int xDistance = std::abs(x1 - x2); //calculate the distance between the aircrafts (X position)
                int yDistance = std::abs(y1 - y2); //calculate the distance between the aircrafts (Y position)
                int zDistance = std::abs(z1 - z2); //calculate the distance between the aircrafts (Z position)

                bool xClose = xDistance <= 3000; // check if the distance is safe or not
                bool yClose = yDistance <= 3000; // check if the distance is safe or not
                bool zClose = zDistance <= 1000; // check if the distance is safe or not

                if (xClose && yClose && zClose) {  // if all three condition fail their check (thus safety violation occurs)
                    printTimestamp();
                    std::cout << "Collision detected at t=" << t << " seconds between Aircraft " << id1 << " and Aircraft " << id2 << "\n";
```

This is our core methodology to find the safety violation in real-time, since the Computer System gets updated with fresh data every second. The next section is involved in solving the safety violation. We designed a structure called collision_msg, which contains the identification number of both unsafe aircrafts and sends it to the Operator Console with the MsgSend function, using a Mutex lock to avoid overwhelming the operator with dozens of requests at once.

Had we had more time, we would’ve liked to implement a feature where the Operator Console runs on many threads, similarly to Aircrafts, where a single request from Computer System would open a single thread, so that the Operator Console could open many threads at once. The Operator will send a character back, either Y for yes (as in intervene and contact the Aircrafts) or N for no (leave

things as is). Upon receiving a response to resolve the safety violation, we will post a synchronization semaphore (used with Communication System) and unlock the Operator Console mutex.

```
operatorConsoleMutex.lock(); // Prevent flooding Operator Console

collision_msg msg;
msg.aircraft1_id = id1;
msg.aircraft2_id = id2;

collision_msg reply;
int status = MsgSend(coid, &msg, sizeof(msg), &reply, sizeof(reply)); // send message to operator
if (status == -1) {
    perror("MsgSend to OperatorConsole failed");
} else {
    if (reply.decision == 'Y') { // receive response from operator
        commData->aircraft1_id = id1;
        commData->aircraft2_id = id2;
        commData->NotifyAircrafts = true;
    }
    sem_post(sem_sync); // used with Communication System
}

operatorConsoleMutex.unlock();
```

The next step is to update our Data display, we will run through the writeToDataDisplay() function (using the writer/reader approach) and use sem_post(sem_display) to notify the DataDisplay process that new data is ready to be processed.

To finish this section, we want to highlight a drawback of our code when it comes to real time execution. The one feature that was required but we were not able to add was specifying a load level in only one process's console and have it automatically set that level in the other consoles.

In other words, Aircraft.cpp will ask the user for the load level (low medium high overload) at the start of runtime because it needs to know what size to make the vector that holds the information of the aircrafts, as well as the size of the memory that needs to be allocated to the shared memory.

Similarly, all the processes that do data processing (Namely Aircrafts, ComputerSystem, CommunicationSystem, Radar and DataDisplay) are required to have their load manually inputted into them at runtime to ensure our code is running as expected. The correct order in which to start the processes are as follows: (1) Radar, (2) OperatorConsole, (3) Computer System, (4) Communication System, (5) Aircrafts, (6) DataDisplay.

D - DataDisplay.cpp

The fourth component that was designed in this project, and one of the easiest to implement, was the DataDisplay.cpp. Due to the simple writer/reader relation with ComputerSystem, DataDisplay waits for sem_post(sem_display) in ComputerSystem at its sem_wait(sem_display) statement.

```
// Read and display grid data
for (int i = 0; i < 30; ++i) {
    sem_wait(sem_display); // Wait for CompSys to indicate new data

    std::cout << "Displaying data from shared memory...\n";
    printGrid(displayData, numAircrafts); // Visualize aircraft data on the grid dynamically

    sleep(3); // Simulate delay between updates
}
```

Once it enters the print grid function, it reads every position of every plane and prints them into a a 2D grid as shown below:

```
Displaying data from shared memory...
* (A0 - Z: 21000)
* (A1 - Z: 25000)
* (A2 - Z: 30000)
* (A3 - Z: 35000)
* (A4 - Z: 19000)
Zone View (Top-down X/Y perspective):
X : 15000, Y : 115000                                                    X : 115000, Y : 115000
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................
.*...............................................
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................*.
.................................................
.*...............................................
.................................................
.................................................
.................................................
.................................................
.................................................
.................................................
X : 15000, Y : 15000                                                     X : 115000, Y : 15000
```

The Z coordinates are printed about the grid, so that the user can see which aircrafts are at which altitude, then the * character represents an Aircraft. The bottom left of the grid is the 15 000, 15 000 coordinate, the bottom right of the grid is the 115 000, 15 000 coordinate, the top left of the grid is the 15 000, 115 000 coordinate, and the top right of the grid is the 115 000, 115 000 coordinate. The distance is linearly scaled. One thing that we wouldve liked implementing is a system where the flight id would appear next to the * symbol so it would be easier

to track aircrafts. For example, A0 would be next to the * representing Aircraft 0. This was abandoned due to a time constraint and was not essential to the functioning of the global system.

E - OperatorConsole.cpp

The fifth component that was designed in this project was the OperatorConsole.cpp. Admittedly speaking, this component was the most confusing to implement because it had to be tied to decision making and resolving the safety violation. Here, we had many ways to approach this situation: the operator could have manually entered new speeds in X Y Z for both planes to directly control what the planes do, but we decided against it because it would be a nightmare to control for the operator to physically get notified many times in a short span of time, where they are responsible for manually entering data tailored to every interaction. In exchange, we designed a Yes/No system that would alter the Z position of the Aircraft.

Since the limit for collision is 1000 units, we made one aircraft decrease its altitude but 1000 units (setting a Z speed of -100 for 10 cycles) and the other increase its altitude but 1000 units (setting a Z speed of 100 for 10 cycles). The operator now only has to respond to a yes/no question, which is much more feasible in a real-time setting. Despite this, receiving dozens of alerts at once in the Operator Console was difficult to properly answer so we came up with a compromise; we commented out the logic that asks the user to manually input a decision, to automatically answer with Y instantly upon receiving the request. We are aware that the Operator had to manually answer the requests (which works well when we have 5 aircraft under low load), but we needed a way to properly answer the demand when using high and overload loads. We also needed to log the Operator's command in a text file, so we decided to run a timer that starts at runtime for 90 seconds (to run exactly as long as the aircraft simulation), and when the 90 seconds run out, the Operator prints the history of its operations to the text file. Finally, the Operator acts as a server in the client/server relationship with Computer System. As such, it uses MsgReceive to listen to the Computer System's

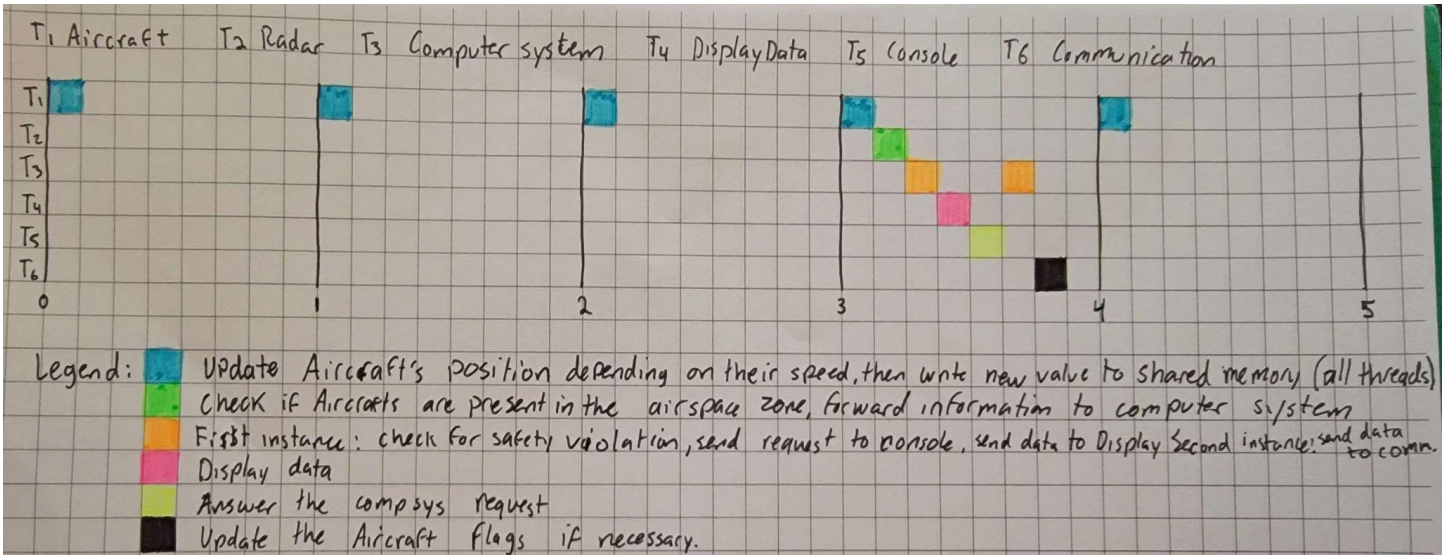
MsgSend command. Upon receiving a request, it will prompt the user (or automatically answer yes) if they want to intervene and notify the aircrafts.

F - CommunicationSystem.cpp

The sixth and final component that was designed in this project was the CommunicationSystem.cpp. This component shares one instance of shared memory (Shm_D) with ComputerSystem and another with Aircraft.cpp (Shm_E). At the start of the execution loop, it waits for ComputerSystem at its sem_wait() statement. Upon receiving the green light from ComputerSystem, it will read in shared memory if any safety violation occurred between two Aircrafts. If a safety violation did occur, it will write in its shared memory with Aircraft.cpp to signal which aircrafts need to modify their flight patterns. To do this, the aircraft with the lowest id will ascend, and its ascension counter will be set to 10. Similarly, the aircraft with the highest id will descend, and its counter will be set to 10. Back in the Aircraft.cpp process, every aircraft will check its own flags to see if they've been notified to either ascend or descend. Upon the counter reaching 0, the ascend and descend flags are reset to false and the Z speed is also set to 0. One of the difficulties that were faced was timing writing the logs into the text files. Especially for the operator console, my execution happened inside the while(true) loop but the lines of code responsible for closing the text file (and saving it) had to be put outside the loop. To get around this issue, we added a timer that would exit the loop after 90 seconds (the runtime of our aircraft).

Lessons Learned

The most important lessons that were learned during this assignment were inter-process communication and designing the architecture. Previously, we never had to make an entire system (composed of many processes) communicate through shared memory. As such, this project taught us how to generate data in a process (namely Aircraft.cpp), forward it to another process (Radar.cpp). The data would then be processed (checking if the airplanes are within the zone or not) before being forwarded again (to Computer System). Upon reaching Computer System, the data would be processed again (checking for safety violations) and respond appropriately depending on the scenario (what to do when there is or isn't a safety violation). From there, we had to decide how to implement the operator console, the data display and the communication system. In the end, we ended up using mostly shared memory (using the reader/writer tutorial as template) and the server/client (using the server/client tutorial as template). If we were able to do this project again, we would focus on adding more multi-threading, especially with the operator console. Because human intervention is required, this task takes a (relatively speaking) extremely long time to finish, which can cause some tasks to meet deadlines. Also, while we were not able to measure the actual execution time of our code, we made an approximation of the relative deadlines.



Conclusion

Generally speaking, the project was mostly a success for several reasons. We were able to properly orchestrate all processes so a self-correcting feedback loop mechanism was successfully integrated in the design. We also fully used inter-process communication through reader/writer and client/server patterns. We would have liked to have a better Data Display and a better safety violation solving algorithm, but we still delivered a functional system given time constraints. The hardest issue was dividing data processing and orchestrating the order of events inside the Computer System. We are aware that a triple for loop is far from ideal but we could not find a better solution. We were also able to properly log all flight information every 20 seconds and log the operator inputs into separate text files. They can both be read in the target file system navigator in QNX Momentics. Another challenge of this project was becoming familiar with QNX, which had a steep learning curve initially. Future work could involve extending the program to outside the zone (take-offs and landings) and could use a more advanced problem solving algorithm. We also could have tried to make the Z position vary a little bit during the flight to mimic high turbulence zones / more realistic flight patterns.

Team Contribution

<u>Code-wise:</u>

Simon:

Aircraft.cpp

ComputerSystem.cpp

Maxime:

Radar.cpp

OperatorConsole.cpp

Patrick:

DataDisplay.cpp

CommunicationSystem.cpp

The structure and layout of our project was discussed during our meetings. Everyone attended every meeting. Simon talked with TAs to understand which interprocess communication were expected (For example, when does a server/client interaction makes sense vs a reader/writer interaction).

Note: while each of the student is responsible for the section below their name, there are overlaps between cpp files due to the nature of the project. For example, Simon and Maxime had to communicate for the shared memory, so Simon did work a bit in Radar and Maxime did work a bit in Aircraft.

<u>Report-wise:</u>

Simon:

Design, Implementation part A and C, Conclusion.

Maxime:

Objective, Introduction, Implementation part B and E.

Patrick:

Title Page, Analysis, Implementation part D and F, Team Contribution.